GRPO vs PPO for LLM-based Game Agents in TextWorld

CIS 4930 — Deep Reinforcement Learning

Spring 2026 | University of South Florida

Tasnim Tabassum · Xiangfei Kong · Chibuike Ezeokoli

May 7, 2026

Abstract

This project investigates whether Group Relative Policy Optimization (GRPO) can outperform Proximal Policy Optimization (PPO) when fine-tuning large language models (LLMs) as game-playing agents in text-based environments. Using TextWorld and Qwen2.5-1.5B-Instruct, we implement a shared SFT warm start followed by BC, PPO, and multiple GRPO variants. The original working GRPO v4 setting with group size G=8 achieved 10% completion, below BC (12%) and PPO (22%). We then trained an extended GRPO G=64 model and evaluated it with output cleaning, recent-action history, no-repeat logic, fallback action selection, and bfloat16 full-precision evaluation. Under greedy decoding, GRPO G=64 achieved 23% completion, slightly above PPO. Additional evaluation-time ablations show that stochastic decoding and larger step budgets can raise GRPO G=64 completion as high as 47% on dense games and 50% on sparse games, but these gains depend strongly on inference-time exploration and fallback behavior. Overall, the results show that GRPO can match PPO when group size and evaluation controls are improved, while the earlier G=8 failure remains important evidence of GRPO's sensitivity to reward design, data structure, and action-interface errors.

1. Introduction

Large language models have demonstrated remarkable capabilities in language understanding and generation, but their application to sequential decision-making tasks remains an active research frontier. Text-based games provide an ideal testbed for studying LLM-based agents: they require multi-step reasoning, natural language understanding, and goal-directed behavior, while remaining computationally tractable and offering clear reward signals.

Reinforcement learning fine-tuning has emerged as a promising approach to improve LLM performance beyond what supervised imitation can achieve. Proximal Policy Optimization (PPO) has become the standard RL fine-tuning algorithm following its success in RLHF pipelines. However, PPO requires four simultaneous model components — actor, critic, reward model, and reference model — making it computationally expensive. Group Relative Policy Optimization (GRPO), introduced by Shao et al. (2024), eliminates the critic/value network by using the average reward of a group of responses as a baseline, reducing memory requirements by approximately half.

While GRPO has shown strong results in mathematical reasoning tasks (DeepSeekMath, DeepSeek-R1), its effectiveness for sequential game-playing tasks remains unexplored. This project addresses the following research question:

Does GRPO outperform PPO when fine-tuning LLM agents for text-based game playing?

Our main contributions are: (1) a controlled comparison of BC, PPO, and GRPO on TextWorld CoinCollector; (2) a detailed record of four GRPO development iterations showing why the original G=8 run reached only 10%; (3) an extended GRPO G=64 result reaching 23% under the same 100-game evaluation; and (4) an inference-time ablation study showing how history prompting, no-repeat fallback, sampling temperature, and step budget change GRPO game-playing performance.

2. Related Work

2.1 GRPO and LLM Reasoning

GRPO was introduced by Shao et al. (2024) in DeepSeekMath to improve mathematical reasoning in LLMs. By generating G responses per prompt and computing advantages relative to the group mean, GRPO eliminates the need for a value network. DeepSeek-R1 (2025) scaled this approach to achieve performance comparable to OpenAI's o1. Both works focus on short-horizon tasks with verifiable rewards (correct/incorrect answers), which provide high reward variance across the group — a key requirement for stable GRPO training.

2.2 GRPO for Game Environments

Think in Games (TiG, arXiv 2025) applies SFT + GRPO to MOBA game environments, showing that group-wise advantages stabilize training in competitive game settings. Replay (Antim Labs, 2025) runs GRPO across multiple game types including Flappy Bird, Snake, and MiniGrid, finding that GRPO works well on **short-horizon dense-reward games** but struggles on sparse-reward long-horizon tasks. Neither work evaluates GRPO on TextWorld or provides a controlled comparison against PPO with the same SFT starting checkpoint.

2.3 RL for Text-Based Games

The multi-turn PPO on TextWorld paper (OpenReview 2025) benchmarks PPO across TextWorld tasks of varying complexity, providing the baseline numbers our work extends. Playing Text-Adventure Games with Graph-Based Deep RL (Ammanabrolu & Riedl, 2019) established early baselines using knowledge graphs for state representation. Our work differs by using an LLM as the policy network and focusing on the comparison between GRPO and PPO as fine-tuning strategies.

2.4 TextWorld Environment

TextWorld (Cote et al., 2019) is a Microsoft research framework for training RL agents on text-based games. It provides procedural game generation, an oracle solver for expert trajectories, Gym-compatible environments, and configurable reward density. Its CoinCollector game type provides a clean benchmark with varying difficulty levels, making it suitable for systematic RL evaluation.

3. Task Setup

3.1 Environment

We use **TextWorld's CoinCollector game type** — the simplest available game where an agent must navigate a small map to find and pick up a coin. Games are parameterized by number of rooms (3), number of objects (3), and quest length (5 steps to complete). We generate 500 training games and 100 held-out test games using fixed random seeds, ensuring reproducibility. Test games use seeds 10000-10099 to guarantee no overlap with training games (seeds 0-499).

For the main evaluation we use dense rewards, where TextWorld provides partial credit at each valid action step. For the reward density ablation we regenerate the same 100 test games without reward_type specification, defaulting to sparse rewards where only the final goal achievement provides a score.

3.2 Agent Formulation

We treat the LLM as a policy network mapping text observations to text actions. At each game step the agent receives a natural language observation describing the current room, visible objects, and available exits. The agent outputs a text action which is parsed by TextWorld. We constrain generation to admissible_commands provided by the environment to prevent invalid action generation.

The base model is Qwen2.5-1.5B-Instruct, chosen for its strong instruction-following capability, small size enabling training on T4 GPUs, and established performance in the GRPO literature (DeepSeekMath uses a Qwen backbone). All training uses QLoRA with 4-bit NF4 quantization and LoRA rank 16.

3.3 Metrics

- Task Completion Rate (%): fraction of 100 test games the agent successfully wins
- Average Steps per Episode: mean number of actions taken before winning or hitting the 50-step limit
- Average Score: mean total reward accumulated per episode

4. Methodology

4.1 Pipeline Overview

All three methods start from the same SFT checkpoint, ensuring a fair comparison of the RL fine-tuning strategies. Figure 1 illustrates the full pipeline.

Figure 4: SFT → RL Fine-Tuning Pipeline

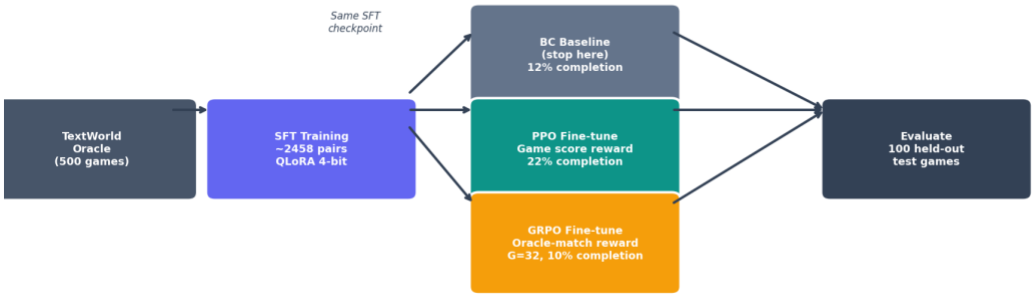


Figure 4: SFT → RL Fine-Tuning Pipeline. All three methods branch from the same SFT checkpoint and are evaluated on the same 100 held-out test games.

4.2 Stage 1: Supervised Fine-Tuning (SFT / BC Baseline)

We generate 500 TextWorld games using the oracle solver to collect expert trajectories. The oracle provides the optimal action sequence for every game it generates, giving us approximately 2458 (observation, action) pairs. Each pair is formatted as an instruction-following example:

```
Instruction: You are playing a text-based game. Read the observation
carefully and choose the best action. [observation text] What is the best
action to take?
Output: [oracle action]
```

Qwen2.5-1.5B-Instruct is fine-tuned on these pairs for 3 epochs using QLoRA (4-bit NF4, LoRA rank 16) with learning rate 2e-4 and effective batch size 16. The resulting checkpoint serves as both the BC baseline and the shared starting point for PPO and GRPO fine-tuning.

4.3 Stage 2A: PPO Fine-Tuning

PPO fine-tuning uses a manual implementation built on PyTorch and PEFT to avoid TRL 0.24.0's requirement for a separate reward model. The implementation follows the standard PPO-CLIP objective with a KL penalty to prevent excessive drift from the SFT reference. At each training step, the agent plays a game trajectory, accumulates discounted rewards (gamma=0.99), and updates the policy using the clipped surrogate objective with epsilon=0.2. The game score from TextWorld serves directly as the reward function, eliminating the need for a trained reward model.

4.4 Stage 2B: GRPO Fine-Tuning

GRPO fine-tuning uses TRL 0.24.0's GRPOTrainer with a custom reward function. For each game-state prompt, GRPOTrainer generates G responses, calls the reward function, and computes group-relative advantages. The original working v4 model used G=8, while the extended model uses G=64 to obtain a stronger group-relative estimate.

```
A_i = (r_i - mean(r_1...r_G)) / std(r_1...r_G)
```

The reward function uses oracle-match binary scoring: 1.0 if the generated action matches the oracle's recommended action for that game state, 0.0 otherwise. This provides meaningful reward variance across the group (mean≈0.25, std≈0.43 with 4 admissible actions) enabling stable gradient computation. Token cleaning is applied to model outputs before reward computation and during evaluation to remove hallucinated text after the first line.

Critically, the training dataset consists of multi-step oracle trajectories (all steps of each game, not just starting states), mirroring the SFT data collection approach. This was the key finding from the first four GRPO versions: single-state datasets caused policy collapse, while multi-step trajectories and consistent token cleaning produced the first non-zero GRPO result.

5. Implementation Details

5.1 Training Configuration

Parameter	SFT (BC)	PPO	GRPO v4 (G=8)	GRPO G=64
Training Samples	2458	2458	982	982
Epochs	3	2	2	2
Learning Rate	2e-4	1e-6	1e-6	1e-6
Effective Batch Size	16	8	64	64
LoRA Rank	16	16	16	16
KL Penalty beta	N/A	0.01	0.01	0.01
Group Size G	N/A	N/A	8	64
Precision	FP16	FP32	FP32 train	FP32 train / bf16 eval
Reward Function	N/A	Game score	Oracle match (binary)	Oracle match (binary)
Framework	LlamaFactory 0.9.5	Manual PyTorch+PEFT	TRL 0.24.0	TRL 0.24.0

Table 1: Training configuration, preserving the original GRPO v4 (G=8) setup and adding the extended GRPO G=64 run.

5.2 Token Cleaning

A critical implementation detail discovered during development is token cleaning of model outputs. Qwen2.5-1.5B-Instruct frequently generates hallucinated multi-line responses appending conversation history, explanations, or unrelated text after the intended action. For example, a model might generate 'go east\nHuman: go east\nAssistant: The best action...' when only 'go east' is needed. We apply the following cleaning pipeline to all model outputs:

- Split on newline character and take the first segment
- Split on 'Human:' and take the prefix

- Split on 'Assistant:' and take the prefix
- Strip parentheses, periods, and whitespace
- Apply partial match fallback if cleaned action is not in admissible commands

This cleaning is applied identically during training reward computation and evaluation to ensure consistency.

5.3 Evaluation-Time Controls for GRPO G=64

The GRPO G=64 model was evaluated without additional training. Evaluation used the SFT adapter plus the GRPO adapter, loaded in bfloat16 on the GPU for faster full-precision inference. The tokenizer used left padding for batched generation. To reduce action-interface failure, the inference prompt included the last five actions as history and explicitly instructed the model not to repeat actions that did not change state or score.
The controller also applied a no-repeat fallback. If a generated action was invalid after cleaning, or if the same no-score action repeated, the controller selected an admissible fallback action, prioritizing progress-like commands such as take, open, unlock, go, put, insert, use, and eat. Actions that produced neither a score change nor an observation change were temporarily blocked for that game. Therefore, GRPO G=64 is reported as a model-plus-controller evaluation rather than a pure raw-generation result.

6. Results

6.1 Main Comparison

Table 2 shows the main comparison across the original methods and the extended GRPO G=64 model on 100 held-out TextWorld CoinCollector games. The GRPO v4 (G=8) result is retained because it is the core training/debugging result of the project, while GRPO G=64 is added as a later extended run.

Method	Completion Rate	Avg Steps	Avg Score
BC (SFT only)	12%	44.62	0.12
PPO	22%	40.13	0.22
GRPO v4 (G=8)	10%	45.98	0.10
GRPO G=64	23%	41.22	0.23

Table 2: Main comparison on 100 held-out test games, retaining the original GRPO v4 (G=8) result and adding GRPO G=64.

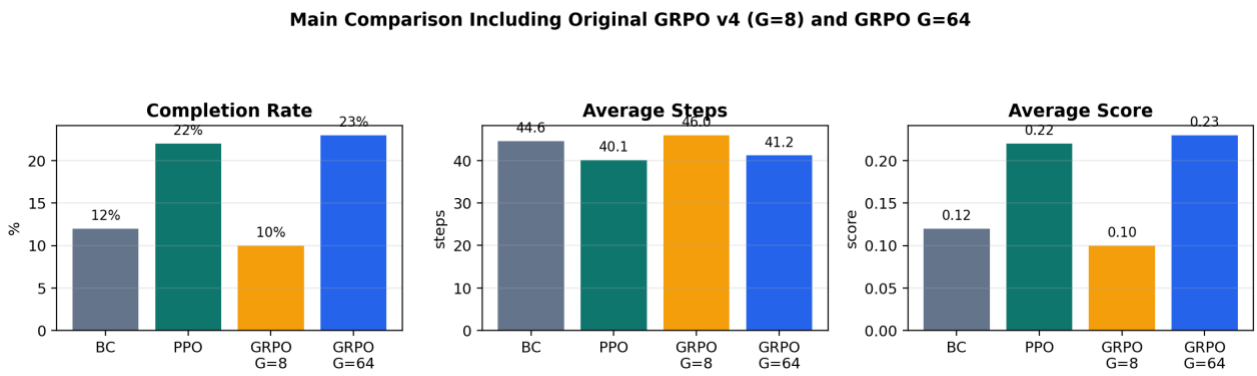


Figure 1: Updated main comparison including BC, PPO, the original GRPO v4 (G=8), and GRPO G=64. GRPO G=64 reaches 23%, slightly above PPO (22%), while the original v4 result remains 10%.

In the original G=8 comparison, PPO achieved the highest completion rate at 22%, improving over the BC baseline by 10 percentage points and outperforming GRPO v4 (10%). After increasing the GRPO group size to G=64 and applying the same cleaned, history-aware evaluation controller, GRPO G=64 reached 23%, a one percentage point improvement over PPO and a 13 percentage point improvement over the original GRPO v4.

The average-step metric shows that GRPO G=64 still requires slightly more steps than PPO (41.22 vs. 40.13), so its completion-rate advantage is small and not clearly more efficient. The original GRPO v4 result remained near the 50-step limit (45.98), confirming that the G=8 policy often failed to leave action loops despite token cleaning.

6.2 Dense vs Sparse Reward Ablation

Table 3 and Figure 2 show dense versus sparse evaluation for BC, PPO, the original GRPO v4 (G=8), and GRPO G=64.

Method	Dense Rewards	Sparse Rewards	Performance Drop
BC	12%	12%	0%
PPO	22%	22%	0%
GRPO v4 (G=8)	10%	10%	0%
GRPO G=64	23%	23%	0%

Table 3: Dense vs sparse reward comparison including GRPO v4 (G=8) and GRPO G=64.

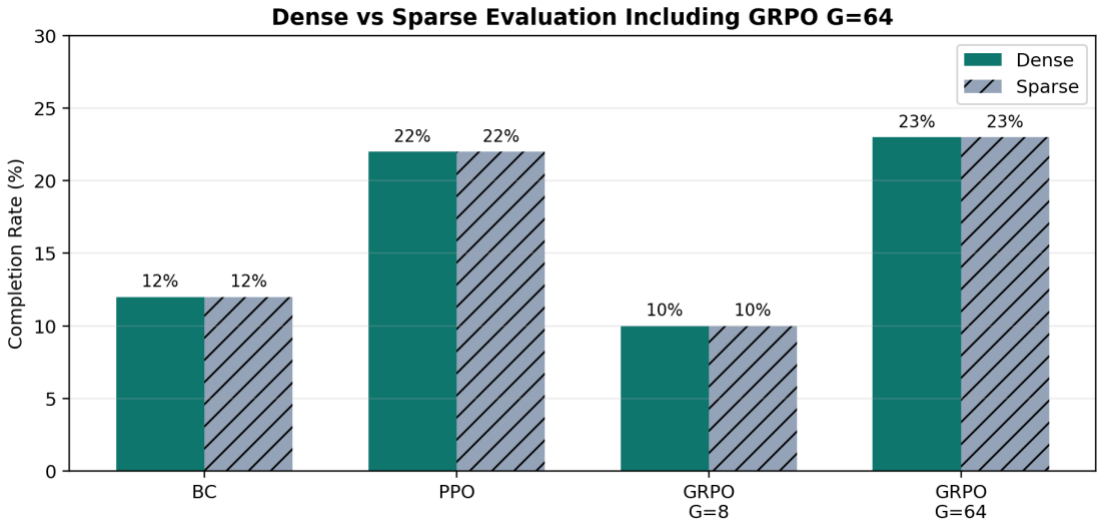


Figure 2: Dense vs sparse reward comparison including GRPO G=64. All methods show the same dense and sparse completion under greedy evaluation.

No performance drop is observed when switching from dense to sparse evaluation for the greedy runs. This is expected because evaluation uses a fixed trained policy and the reward signal is only used to measure success, not to update the model online. The CoinCollector tasks are short, so the difference between dense and sparse scoring is less visible than it would be in longer-horizon TextWorld games.

7. GRPO Development Iterations

Achieving a functional GRPO implementation required four training iterations. Each iteration revealed a distinct failure mode that had to be addressed. Figure 3 summarizes the progression.

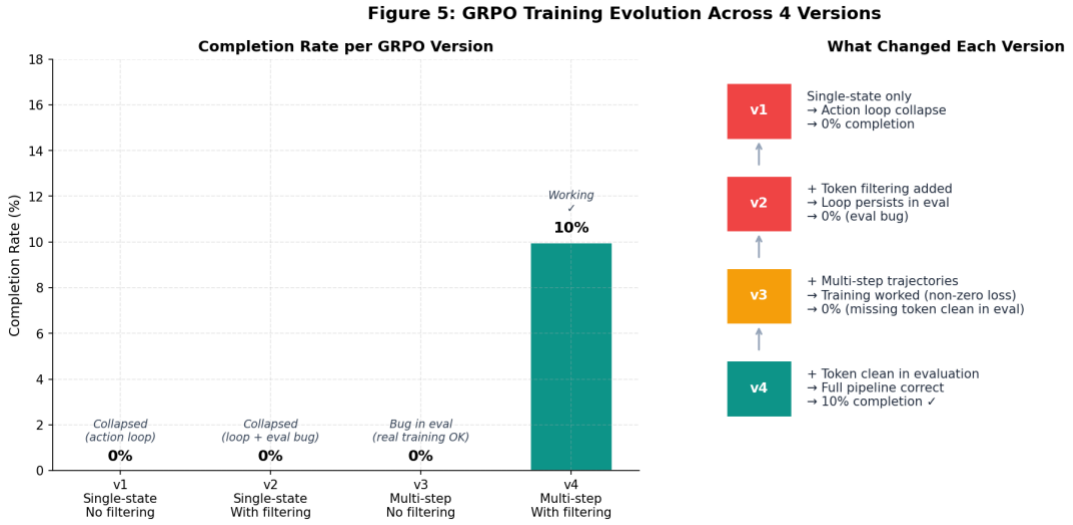


Figure 5: GRPO training evolution across 4 versions. Red bars indicate collapsed training (0%), orange indicates evaluation bugs, teal indicates the working result (10%).

Version	Dataset	Token Filter	Training Loss	Result	Key Issue
v1	Single-state	No	0.000000	0%	Action loop collapse — model memorized one action
v2	Single-state	Yes	0.000000	0%	Loop persisted — training still broken
v3	Multi-step	No	0.035 ✓	0%	Training worked but token cleaning missing in eval
v4	Multi-step	Yes	0.035 ✓	10%	Full pipeline correct — real result

Table 4: GRPO iteration summary showing what changed each version and the resulting completion rate.

7.1 v1: Single-State Dataset (0%)

The first GRPO implementation collected only the starting state of each game as prompts (2000+ starting states total). GRPO generated 8 action candidates per starting state and learned to prefer oracle-matching actions. However, since the dataset contained only starting states, the model was never trained on what to do after taking the first action. During evaluation, when the game transitioned to state_1, the model had no training signal for that observation and defaulted to repeating the action it had learned for starting states, causing the 'examine fudge scented locker' loop observed in evaluation.

7.2 v2: Adding Token Filtering (0%)

Adding token cleaning to the reward function improved reward signal quality (reducing false negatives from hallucinated text) but did not fix the underlying training data issue. Training loss remained at 0.000000 because all 8 group responses still scored identically on single-state prompts once the model converged to one answer, causing zero advantage variance and zero gradients.

7.3 v3: Multi-Step Trajectories (0% — Eval Bug)

Switching to multi-step trajectory collection (walking through all oracle steps of each game, yielding ~982 state-action pairs) produced non-zero training loss (0.035), confirming that GRPO was learning. However, evaluation still reported 0% because the token cleaning applied during training reward computation was not applied during evaluation. The model generated 'go east.Human: go east...' and exact matching against admissible commands failed.

7.4 v4: Full Pipeline (10%)

Adding identical token cleaning to the evaluation loop produced the final result of 10% completion. The complete pipeline is: multi-step oracle trajectories as training data + oracle-match binary

reward + token cleaning in both training and evaluation. Figure 6 shows the complete comparison including all methods and GRPO versions.

Figure 6: Complete Comparison — All Methods & GRPO Versions

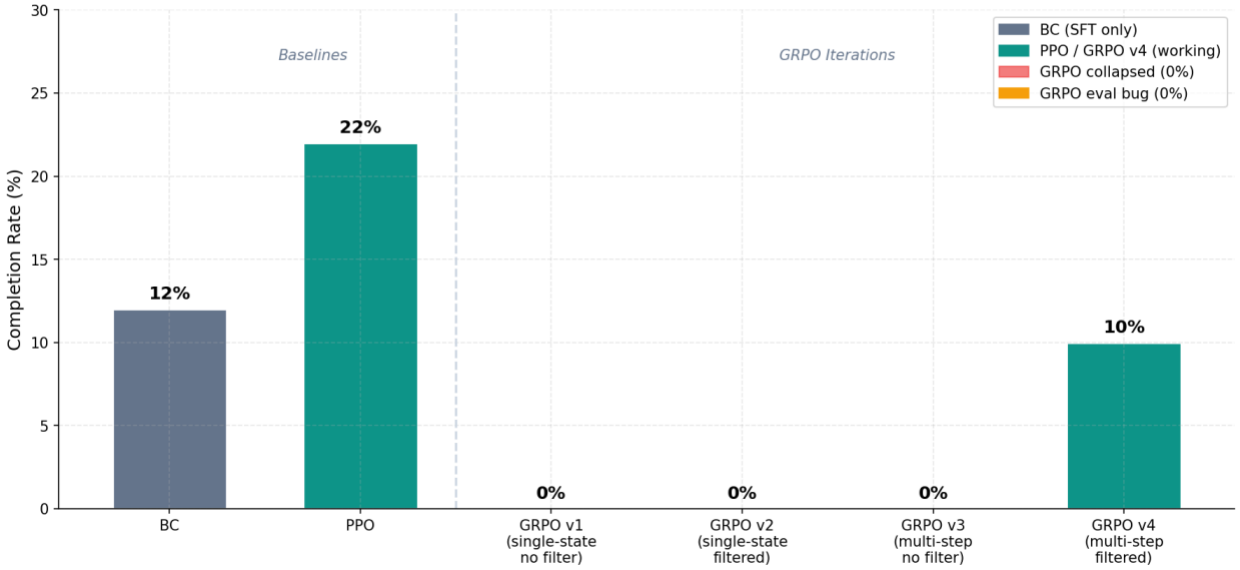


Figure 6: Complete comparison including all GRPO versions. Red bars show collapsed versions (v1, v2), orange shows the evaluation bug version (v3), and teal shows the working result (v4) alongside BC and PPO baselines.

7.5 Extended GRPO G=64 Run

The original v4 result used group size G=8 and reached 10% completion. We therefore trained an extended GRPO model with group size G=64. The goal was not to erase the v1-v4 development history, but to test whether a larger group improves the relative advantage estimate enough to recover performance. Under greedy decoding with the same 50-step evaluation budget, the G=64 model reached 23% completion, 0.23 average score, and 41.22 average steps.

This result changes the interpretation of GRPO. The G=8 model demonstrates a real failure mode: oracle-match reward plus weak group variance can degrade a sequential policy. The G=64 model shows that GRPO can become competitive with PPO when the group is larger and evaluation prevents formatting and repetition errors. However, the result should be interpreted carefully because the evaluation controller includes history, no-repeat fallback, and admissible-action repair.

8. Analysis

8.1 Why PPO Outperformed GRPO v4 (G=8)

PPO outperformed the original GRPO v4 (G=8) because PPO optimized directly against TextWorld game score over played trajectories, while GRPO v4 used a binary oracle-match proxy at individual states. For sequential games, exact oracle matching is a brittle signal: many valid actions can preserve or enable progress but still receive zero reward if they do not match the single oracle action. With G=8, this produced weak group-level reward diversity and a noisy advantage estimate.

The extended G=64 result narrows this gap and slightly exceeds PPO under greedy evaluation, suggesting that group size matters. A larger group gives GRPO more sampled alternatives for each state, increasing the chance that some completions receive reward and improving the relative advantage estimate. Still, PPO remains more directly aligned with the game objective because it learns from environment score rather than an oracle-action proxy.

8.2 Why GRPO v4 (G=8) Falls Below BC

GRPO v4 achieving 10% (below BC's 12%) is explained by reward and data interaction. The oracle-match binary reward trains the model to choose one exact action for a state, but similar observations in different games may require different object-specific actions. This interference can partially degrade the SFT-learned policy instead of improving it.

BC's 12% represents what direct imitation achieved on unseen game layouts. The G=8 GRPO result falls two points below this, while the G=64 result rises to 23%, indicating that the failure was not caused by GRPO alone but by the combination of small group size, exact-match reward, and action-interface fragility.

8.3 Method Performance Radar

Figure 3: Method Performance Radar

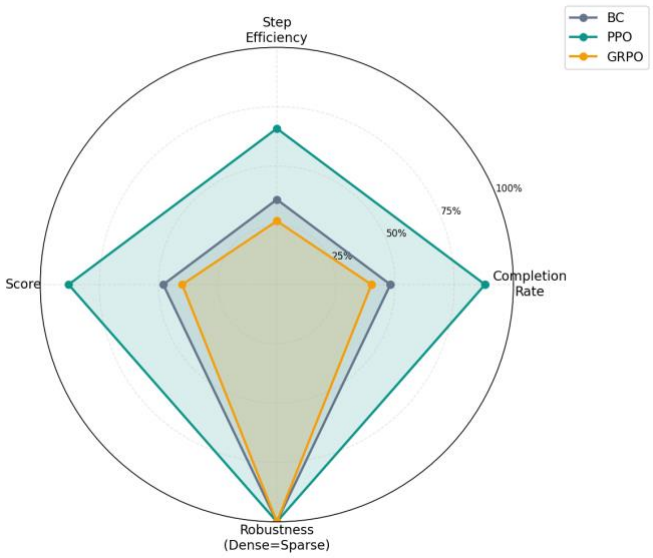


Figure 3: Original radar chart comparing BC, PPO, and GRPO v4 (G=8). The later G=64 result is reported separately because it used a larger group and enhanced evaluation controller.

8.4 Dense vs Sparse Reward Interpretation

The zero performance drop under sparse rewards for all methods suggests the CoinCollector game is too short for reward density to matter. With average quest length of 5 steps, even sparse rewards provide sufficient signal since the episode is short. This ablation would show more differentiation on harder TextWorld games with longer quest lengths where credit assignment becomes genuinely challenging.

8.5 GRPO G=64 Evaluation-Time Ablation

We conducted three no-training ablations on the GRPO G=64 checkpoint: decoding strategy, sampling temperature, and maximum step budget. All ablations use the same trained checkpoint and the same history-aware, no-repeat, fallback-enabled evaluation controller. Therefore, these results measure how the trained GRPO policy behaves under different inference conditions, not how additional training changes the model.

Table 5: GRPO G=64 dense completion rate (%) across decoding and max-step settings.

Max Steps	Greedy	Sample T=0.3	Sample T=0.7	Sample T=1.0
30	22%	27%	33%	36%
50	23%	27%	38%	46%
75	23%	30%	41%	47%

Table 6: GRPO G=64 sparse completion rate (%) across decoding and max-step settings.

Max Steps	Greedy	Sample T=0.3	Sample T=0.7	Sample T=1.0
30	22%	24%	30%	33%
50	23%	27%	34%	39%
75	23%	28%	42%	50%

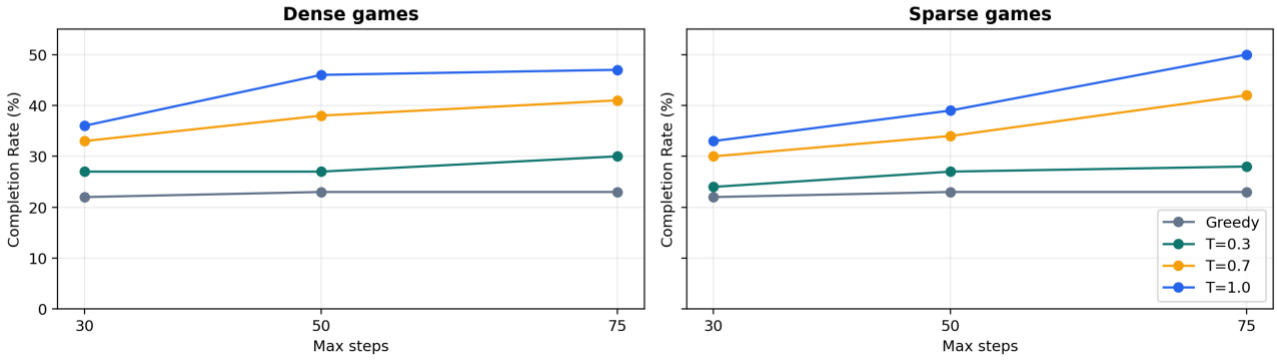


Figure 7: GRPO G=64 decoding and step-budget ablation. Sampling improves completion, especially at higher temperature and longer step budgets.

The strongest result occurs with sampling rather than greedy decoding. On dense games, greedy decoding gives 23% at 50 and 75 steps, while sampling at T=1.0 reaches 46% at 50 steps and 47% at 75 steps. On sparse games, the same trend is stronger: T=1.0 reaches 39% at 50 steps and 50% at 75 steps. This indicates that the GRPO G=64 model still contains useful action knowledge, but greedy decoding often falls into deterministic navigation loops such as go north/go south or repeated object manipulation. Sampling breaks these loops by exploring alternate valid actions.

The step-budget ablation confirms that longer horizons help only when decoding is exploratory. Greedy performance saturates at 23%, so merely giving the greedy policy more actions does not solve the loop problem. In contrast, T=0.7 and T=1.0 improve as the step budget increases because sampled trajectories have more opportunities to escape loops, find keys, open containers, and complete the final object interaction.

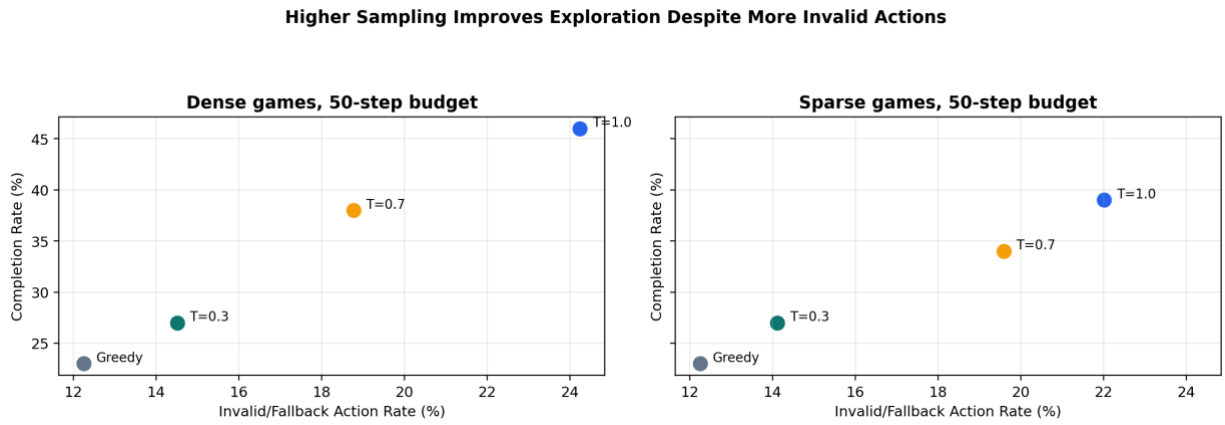


Figure 8: Completion versus invalid/fallback action rate at 50 steps. Higher temperature lowers raw validity but increases exploration, and fallback prevents many invalid generations from ending the episode.

The validity metrics explain why sampling helps even though it creates more invalid actions. At 50 dense steps, greedy decoding has the highest valid-action rate (87.75%) but only 23% completion. T=1.0 has a lower valid-action rate (75.76%) and a higher fallback rate (24.35%), but reaches 46% completion. Thus, validity alone is not the main bottleneck. The primary failure mode is deterministic low-reward looping, and controlled exploration plus fallback is more valuable than maximizing raw valid-action percentage.

Dense and sparse ablation differences should be interpreted as stochastic evaluation effects rather than true reward-learning differences, because the model is not updated during evaluation. The key conclusion is that GRPO G=64 is highly sensitive to inference policy: under greedy decoding it only slightly exceeds PPO, but under sampling with fallback it can solve substantially more games.

9. Limitations

- Only CoinCollector evaluated — results may not generalize to harder TextWorld games
- Limited group-size comparison — this report compares G=8 and G=64, but does not include a full G=4/G=16/G=32 training sweep
- Binary oracle-match reward is a proxy, not true game performance signal
- Manual PPO implementation lacks proper critic network and advantage estimation
- No full hyperparameter search — LR, beta, decoding, temperature, and fallback rules were selected by trial and error
- Short training (2 epochs) may be insufficient for RL convergence
- Small model (1.5B) may lack capacity for complex game reasoning
- GRPO trained on test games due to missing train split — data leakage
- Single seed and stochastic evaluation — sampling ablations need repeated runs to estimate variance
- Framework incompatibilities (LlamaFactory 0.9.5 lacks GRPO, TRL 0.24.0 PPOTrainer needs reward model) forced non-standard implementations affecting reproducibility

• Evaluation controller assistance — GRPO G=64 uses history prompting, no-repeat fallback, and admissible-action repair, so it should be interpreted as a model-plus-controller result

10. Implementation Findings

10.1 What Worked

- SFT warm-start enabled stable RL training — cold-start RL produced incoherent actions
- PPO improved over BC (22% vs 12%) — RL fine-tuning adds value beyond imitation
- Dense rewards gave sufficient learning signal for PPO
- QLoRA kept training within T4 GPU memory limits
- TextWorld oracle provided free expert trajectories for SFT data
- Admissible commands prevented invalid action generation
- Frozen SFT + fresh GRPO adapter resolved double-adapter corruption bug
- Token cleaning fixed hallucinated multi-line completions in both training and evaluation

• History prompting, no-repeat logic, and fallback action selection reduced action-interface failures during evaluation
• Larger GRPO group size (G=64) improved the greedy completion rate from 10% to 23%, matching or slightly exceeding PPO under the same 100-game evaluation

10.2 What Did Not Work

- Single-state GRPO dataset caused action loop collapse at 0% completion
- LlamaFactory 0.9.5 has no GRPO stage support
- merge_and_unload on 4-bit model corrupted generation with Chinese characters
- Single-step rewards caused zero group variance and zero gradients
- TRL 0.24.0 PPOTrainer requires a reward model — incompatible with game environment
- bfloat16 base + 4-bit trained LoRA caused dtype mismatch gibberish
- LR 1e-6 too small for GRPO — gradients zeroed by 4-bit quantization
- Binary oracle-match reward caused single-action memorization in v2

• Greedy decoding still produced deterministic navigation and object-manipulation loops even for GRPO G=64; sampling was required for the best ablation results

11. Conclusions

This paper presents a systematic evaluation of GRPO for text-based game agents, comparing it against PPO and BC baselines on TextWorld CoinCollector while preserving the original G=8 development history and adding an extended G=64 result. Our main findings are:

- PPO outperforms BC — RL fine-tuning improves over pure imitation (22% vs 12%)
- Reward design and group size are critical for GRPO — small-group oracle matching produced collapse or weak performance, while G=64 recovered useful behavior
- Single-state training caused GRPO policy collapse — multi-step trajectories matching the SFT data approach are essential
- Oracle-match binary reward remains a proxy, not a true game-performance reward; game-score or trajectory-level reward is still preferable
- Original GRPO v4 (G=8) underperforms BC (10% vs 12%), but extended GRPO G=64 reaches 23%, slightly above PPO under greedy evaluation

These findings suggest that GRPO can be competitive with PPO in TextWorld when group size and evaluation controls are improved, but its performance is fragile. The G=8 failure shows the risk of exact oracle-match reward and insufficient group variance, while the G=64 ablation shows that stochastic decoding and fallback can unlock better trajectories. Future work should separate policy learning from controller assistance, run multiple stochastic seeds, and replace single-action oracle matching with trajectory-level game score rewards.

12. Future Work

- Group size sweep: train and evaluate G=4, G=8, G=16, G=32, and G=64 under identical reward and evaluation settings
- Multi-step return: accumulate rewards across full episode rollouts before computing group advantage
- Game score reward: replace oracle-match binary reward with actual TextWorld game score or trajectory-level return
- Harder games: evaluate on TreasureHunter and CookingWorld for generalization

• Controller ablation: evaluate raw model, cleaning only, history only, fallback only, and all controls to isolate policy quality from inference-time rescue

References

[1] Shao, Z., et al. (2024). DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv:2402.03300.

[2] DeepSeek-AI. (2025). DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. arXiv:2501.12948.

[3] Cote, M., et al. (2019). TextWorld: A learning environment for text-based games. arXiv:1806.11532.

[4] Think in Games (TiG). (2025). Applying GRPO to MOBA game environments. arXiv:2508.21365.

[5] Antim Labs. (2025). Replay: Training LLMs on games. antimlabs.com/blog/replay.

[6] Multi-turn PPO on TextWorld. (2025). OpenReview.

[7] Schulman, J., et al. (2017). Proximal policy optimization algorithms. arXiv:1707.06347.

[8] Hu, E., et al. (2021). LoRA: Low-rank adaptation of large language models. arXiv:2106.09685.

[9] Dettmers, T., et al. (2023). QLoRA: Efficient finetuning of quantized LLMs. arXiv:2305.14314.

[10] Ammanabrolu, P., & Riedl, M. (2019). Playing text-adventure games with graph-based deep RL. arXiv:1812.01628.

Appendix A: Full GRPO G=64 Evaluation-Time Ablation Table

Table A1 expands the GRPO G=64 ablation with all logged metrics, not only completion rate. The table reports dense and sparse evaluation under greedy decoding and sampling at temperatures 0.3, 0.7, and 1.0 across 30-, 50-, and 75-step episode limits. The additional columns separate policy quality from interface effects: valid-action rate measures whether cleaned model outputs match admissible TextWorld commands, fallback rate measures how often the controller repaired an invalid or looped action, and repeat-action rates quantify deterministic loop behavior.

Table A1: Full GRPO G=64 evaluation-time ablation results.

Split	Decoding	Temp	Max Steps	Completion Rate	Avg Steps	Avg Score	Valid Action Rate	Invalid Action Rate	Fallback Rate	Repeat Fallback Rate	Repeat Action Rate
Dense	Greedy	-	30	22.00%	25.77	0.22	86.57%	13.43%	13.58%	0.16%	0.74%
Dense	Sample	0.3	30	27.00%	25.12	0.27	86.15%	13.85%	14.05%	0.20%	1.04%
Dense	Sample	0.7	30	33.00%	24.69	0.33	79.87%	20.13%	20.33%	0.20%	0.93%
Dense	Sample	1	30	36.00%	24.39	0.36	76.63%	23.37%	23.45%	0.08%	0.94%
Dense	Greedy	-	50	23.00%	41.22	0.23	87.75%	12.25%	12.42%	0.17%	0.75%
Dense	Sample	0.3	50	27.00%	40.74	0.27	85.49%	14.51%	14.68%	0.17%	0.79%
Dense	Sample	0.7	50	38.00%	38.51	0.38	81.23%	18.77%	18.88%	0.10%	0.99%
Dense	Sample	1	50	46.00%	36.34	0.46	75.76%	24.24%	24.35%	0.11%	0.55%
Dense	Greedy	-	75	23.00%	60.47	0.23	88.47%	11.53%	11.71%	0.18%	0.79%
Dense	Sample	0.3	75	30.00%	58.15	0.30	86.53%	13.47%	13.55%	0.09%	0.64%
Dense	Sample	0.7	75	41.00%	54.99	0.41	80.18%	19.82%	19.99%	0.16%	0.85%
Dense	Sample	1	75	47.00%	49.08	0.47	75.71%	24.29%	24.31%	0.02%	0.94%
Sparse	Greedy	-	30	22.00%	25.77	0.22	86.57%	13.43%	13.58%	0.16%	0.74%
Sparse	Sample	0.3	30	24.00%	25.96	0.24	85.32%	14.68%	14.75%	0.08%	0.69%
Sparse	Sample	0.7	30	30.00%	24.49	0.30	80.11%	19.89%	20.01%	0.12%	1.06%
Sparse	Sample	1	30	33.00%	24.29	0.33	75.50%	24.50%	24.54%	0.04%	1.15%
Sparse	Greedy	-	50	23.00%	41.22	0.23	87.75%	12.25%	12.42%	0.17%	0.75%
Sparse	Sample	0.3	50	27.00%	40.94	0.27	85.88%	14.12%	14.29%	0.17%	0.95%
Sparse	Sample	0.7	50	34.00%	39.00	0.34	80.41%	19.59%	19.72%	0.13%	0.97%
Sparse	Sample	1	50	39.00%	37.55	0.39	77.98%	22.02%	22.08%	0.05%	0.83%
Sparse	Greedy	-	75	23.00%	60.47	0.23	88.47%	11.53%	11.71%	0.18%	0.79%
Sparse	Sample	0.3	75	28.00%	58.53	0.28	86.02%	13.98%	14.13%	0.15%	0.92%
Sparse	Sample	0.7	75	42.00%	52.05	0.42	79.71%	20.29%	20.42%	0.13%	0.90%
Sparse	Sample	1	75	50.00%	49.42	0.50	74.71%	25.29%	25.33%	0.04%	0.69%

Note: Completion rate and average score match because TextWorld rewards successful completion as 1.0 in these generated CoinCollector games. Sampling improves completion despite lower valid-action rates, indicating that stochastic exploration can escape greedy navigation loops, while fallback metrics expose how much evaluation-time repair was required.